

CS2023 Algoritmos y Estructuras de Datos

Árbol binario de búsqueda

Semana 6

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

14 de septiembre de 2026

Objetivos de la sesión



Objetivos de la sesión



- ▶ ¿Qué es un árbol?

Objetivos de la sesión



- ▶ ¿Qué es un árbol?
- ▶ Árboles binarios: definición, tipos y recorridos

Objetivos de la sesión



- ▶ ¿Qué es un árbol?
- ▶ Árboles binarios: definición, tipos y recorridos
- ▶ La propiedad de Árbol Binario de Búsqueda (BST)

Objetivos de la sesión



- ▶ ¿Qué es un árbol?
- ▶ Árboles binarios: definición, tipos y recorridos
- ▶ La propiedad de Árbol Binario de Búsqueda (BST)
- ▶ El TDA Árbol Binario de Búsqueda

Objetivos de la sesión



- ▶ ¿Qué es un árbol?
- ▶ Árboles binarios: definición, tipos y recorridos
- ▶ La propiedad de Árbol Binario de Búsqueda (BST)
- ▶ El TDA Árbol Binario de Búsqueda
- ▶ Complejidad de sus operaciones — y su talón de Aquiles

Motivación



¡Piénsalo!

Ya conocemos arreglos, listas enlazadas, pilas/colas y tablas hash. Si solo necesitas insertar/buscar/ eliminar rápido, ¿la tabla hash no es ya la respuesta perfecta?

Motivación



¡Piénsalo!

Ya conocemos arreglos, listas enlazadas, pilas/colas y tablas hash. Si solo necesitas insertar/buscar/ eliminar rápido, ¿la tabla hash no es ya la respuesta perfecta?

Lo que la tabla hash **no** te da

Una tabla hash no preserva ningún **orden**: no puedes pedir “el mínimo”, “el siguiente mayor que x ”, o “todos los elementos en orden”, sin recorrer **todo** en $O(n)$.

Motivación



¡Piénsalo!

Ya conocemos arreglos, listas enlazadas, pilas/colas y tablas hash. Si solo necesitas insertar/buscar/ eliminar rápido, ¿la tabla hash no es ya la respuesta perfecta?

Lo que la tabla hash **no** te da

Una tabla hash no preserva ningún **orden**: no puedes pedir “el mínimo”, “el siguiente mayor que x ”, o “todos los elementos en orden”, sin recorrer **todo** en $O(n)$.

Hoy: una estructura que combina lo mejor de ambos mundos — operaciones rápidas, **y** orden.

Árboles

¿Qué es un árbol?



Definición (informal)

Un **árbol** es una colección de **nodos** conectados de forma jerárquica: hay un nodo especial, la **raíz**, y cada nodo tiene cero o más **hijos** — nodos “debajo” de él.

¿Qué es un árbol?



Definición (informal)

Un **árbol** es una colección de **nodos** conectados de forma jerárquica: hay un nodo especial, la **raíz**, y cada nodo tiene cero o más **hijos** — nodos “debajo” de él.

¡Piénsalo!

A diferencia de una lista enlazada (donde cada nodo tiene a lo más **un** siguiente), ¿qué gana un árbol al permitir **varios** hijos por nodo?

¿Qué es un árbol?



Definición (informal)

Un **árbol** es una colección de **nodos** conectados de forma jerárquica: hay un nodo especial, la **raíz**, y cada nodo tiene cero o más **hijos** — nodos “debajo” de él.

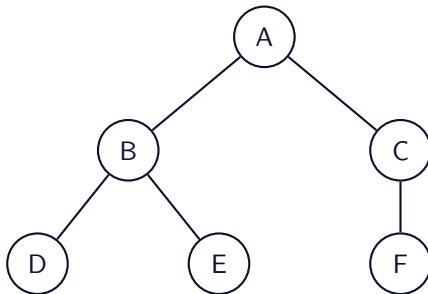
¡Piénsalo!

A diferencia de una lista enlazada (donde cada nodo tiene a lo más **un** siguiente), ¿qué gana un árbol al permitir **varios** hijos por nodo?

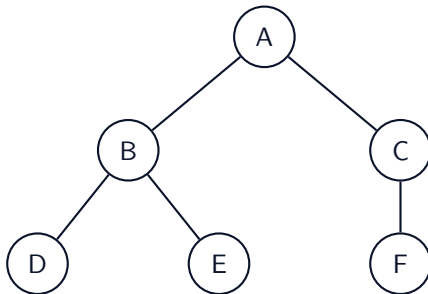
Respuesta

Ramificación: desde la raíz se puede llegar a muchísimos nodos en pocos pasos, sin recorrer todo secuencialmente — la clave para operaciones rápidas.

Terminología



Terminología



A es la **raíz**; **B** es **padre** de D y E; D, E y F son **hojas** (sin hijos); la **profundidad** de D es 2 (aristas desde la raíz); la **altura** del árbol es 2 (la profundidad máxima); el subárbol con raíz B contiene $\{B, D, E\}$.

¿Dónde aparecen los árboles?



En la vida real

Un árbol genealógico, la estructura de carpetas de tu computadora, el índice de un libro (capítulos → secciones → subsecciones).

¿Dónde aparecen los árboles?



En la vida real

Un árbol genealógico, la estructura de carpetas de tu computadora, el índice de un libro (capítulos → secciones → subsecciones).

En cómputo

El DOM de una página web, el AST de un compilador, el árbol de decisión de un juego — y, como veremos en varias clases más, una familia entera de estructuras de datos.

Árboles binarios

La restricción: a lo más 2 hijos



Definición

Un **árbol binario** es un árbol donde cada nodo tiene a lo más **dos** hijos, distinguidos como **izquierdo** y **derecho**.

La restricción: a lo más 2 hijos



Definición

Un **árbol binario** es un árbol donde cada nodo tiene a lo más **dos** hijos, distinguidos como **izquierdo** y **derecho**.

¡Piénsalo!

¿Por qué fijar el número de hijos en exactamente 2, y no 3 o 4?

La restricción: a lo más 2 hijos

Definición

Un **árbol binario** es un árbol donde cada nodo tiene a lo más **dos** hijos, distinguidos como **izquierdo** y **derecho**.

¡Piénsalo!

¿Por qué fijar el número de hijos en exactamente 2, y no 3 o 4?

Respuesta

Con 2 hijos, cada nodo representa una **decisión binaria** (izquierda o derecha) — exactamente lo que necesitamos para una búsqueda tipo “menor o mayor”.

Representación conceptual



Un nodo

Cada nodo guarda: su **dato**, y punteros a su hijo **izquierdo**, hijo **derecho** (y opcionalmente a su **padre**). Si un hijo no existe, el puntero es nulo.

Representación conceptual



Un nodo

Cada nodo guarda: su **dato**, y punteros a su hijo **izquierdo**, hijo **derecho** (y opcionalmente a su **padre**). Si un hijo no existe, el puntero es nulo.

La implementación real la veremos en la próxima clase — hoy nos quedamos en el concepto.

Tipos de árbol binario



Tipos de árbol binario



- ▶ **Lleno** (*full*): todo nodo tiene 0 o 2 hijos, nunca 1.

Tipos de árbol binario



- ▶ **Lleno** (*full*): todo nodo tiene 0 o 2 hijos, nunca 1.
- ▶ **Completo** (*complete*): todos los niveles llenos, salvo quizás el último, que se llena de izquierda a derecha.

Tipos de árbol binario



- ▶ **Lleno** (*full*): todo nodo tiene 0 o 2 hijos, nunca 1.
- ▶ **Completo** (*complete*): todos los niveles llenos, salvo quizás el último, que se llena de izquierda a derecha.
- ▶ **Perfecto**: todos los niveles completamente llenos — $2^{h+1} - 1$ nodos en altura h .

Tipos de árbol binario



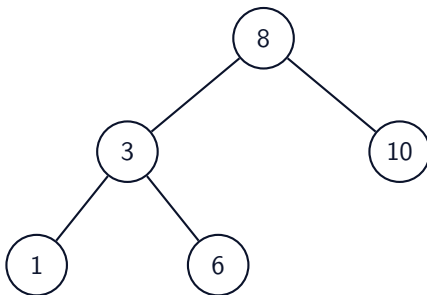
- ▶ **Lleno** (*full*): todo nodo tiene 0 o 2 hijos, nunca 1.
- ▶ **Completo** (*complete*): todos los niveles llenos, salvo quizás el último, que se llena de izquierda a derecha.
- ▶ **Perfecto**: todos los niveles completamente llenos — $2^{h+1} - 1$ nodos en altura h .
- ▶ **Degenerado**: cada nodo tiene a lo más 1 hijo — en la práctica, una lista enlazada disfrazada.

Tipos de árbol binario

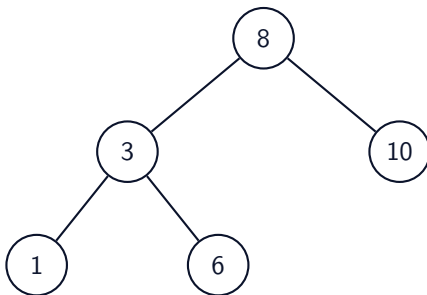
- ▶ **Lleno** (*full*): todo nodo tiene 0 o 2 hijos, nunca 1.
- ▶ **Completo** (*complete*): todos los niveles llenos, salvo quizás el último, que se llena de izquierda a derecha.
- ▶ **Perfecto**: todos los niveles completamente llenos — $2^{h+1} - 1$ nodos en altura h .
- ▶ **Degenerado**: cada nodo tiene a lo más 1 hijo — en la práctica, una lista enlazada disfrazada.

Guarda esa palabra, **degenerado** — va a volver más tarde en esta misma clase.

Recorridos

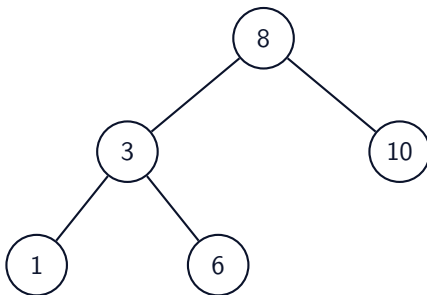


Recorridos



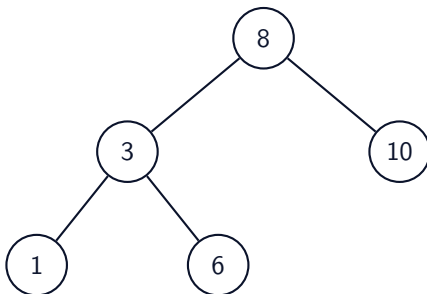
- **Preorder** (raíz, izq., der.): 8, 3, 1, 6, 10

Recorridos



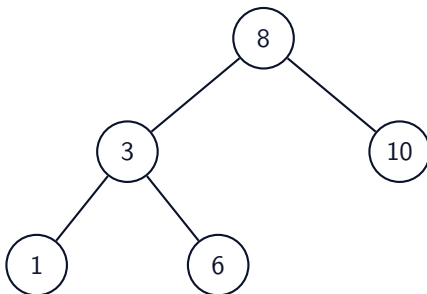
- ▶ **Preorder** (raíz, izq., der.): 8, 3, 1, 6, 10
- ▶ **Inorder** (izq., raíz, der.): 1, 3, 6, 8, 10

Recorridos



- ▶ **Preorder** (raíz, izq., der.): 8, 3, 1, 6, 10
- ▶ **Inorder** (izq., raíz, der.): 1, 3, 6, 8, 10
- ▶ **Postorder** (izq., der., raíz): 1, 6, 3, 10, 8

Recorridos



- ▶ **Preorder** (raíz, izq., der.): 8, 3, 1, 6, 10
- ▶ **Inorder** (izq., raíz, der.): 1, 3, 6, 8, 10
- ▶ **Postorder** (izq., der., raíz): 1, 6, 3, 10, 8

¿Notas algo curioso en el orden **inorder** de este árbol en particular? Guárdalo — no es casualidad, y lo vamos a explicar en un momento.

Árbol Binario de Búsqueda (BST)

La propiedad BST



Definición

Un **Árbol Binario de Búsqueda** es un árbol binario donde, para **todo** nodo x : todo valor en su subárbol izquierdo es $\leq x$, y todo valor en su subárbol derecho es $\geq x$.

La propiedad BST



Definición

Un **Árbol Binario de Búsqueda** es un árbol binario donde, para **todo** nodo x : todo valor en su subárbol izquierdo es $\leq x$, y todo valor en su subárbol derecho es $\geq x$.

¡Piénsalo!

Con esta propiedad, buscar un valor v desde la raíz: en cada nodo, ¿cuánta información te da **una** comparación?

La propiedad BST



Definición

Un **Árbol Binario de Búsqueda** es un árbol binario donde, para **todo** nodo x : todo valor en su subárbol izquierdo es $\leq x$, y todo valor en su subárbol derecho es $\geq x$.

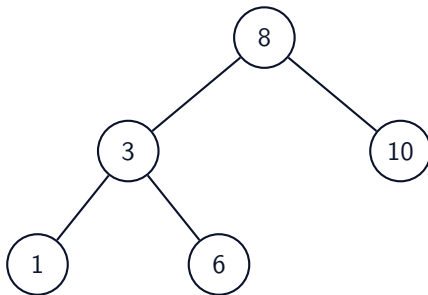
¡Piénsalo!

Con esta propiedad, buscar un valor v desde la raíz: en cada nodo, ¿cuánta información te da **una** comparación?

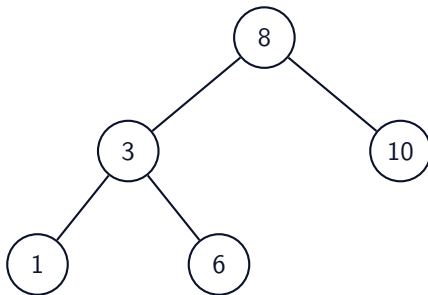
Respuesta

Te dice de qué **lado** está v (si no es el nodo actual) — exactamente como la búsqueda binaria sobre un arreglo ordenado, pero **sin** necesitar memoria contigua.

El árbol de antes, ahora como BST



El árbol de antes, ahora como BST



8: izquierda $\{3, 1, 6\}$ (todo ≤ 8), derecha $\{10\}$ (todo ≥ 8). 3: izquierda $\{1\}$, derecha $\{6\}$. **Sí** es un BST válido.

La revelación: inorder da orden



Lo que ya notaste

El recorrido inorder de este árbol dio 1, 3, 6, 8, 10 — **exactamente** ordenado.

La revelación: inorder da orden



Lo que ya notaste

El recorrido inorder de este árbol dio 1, 3, 6, 8, 10 — **exactamente** ordenado.

¡No es casualidad!

Para **cualquier** BST, un recorrido inorder visita los elementos en orden creciente — es consecuencia directa e inevitable de la propiedad BST en cada nodo.

La revelación: inorder da orden



Lo que ya notaste

El recorrido inorder de este árbol dio 1, 3, 6, 8, 10 — **exactamente** ordenado.

¡No es casualidad!

Para **cualquier** BST, un recorrido inorder visita los elementos en orden creciente — es consecuencia directa e inevitable de la propiedad BST en cada nodo.

Un BST no es solo “rápido de buscar”: también sabe **ordenarse a sí mismo**, gratis.

El TDA Árbol Binario de Búsqueda



Operaciones

El TDA Árbol Binario de Búsqueda



Operaciones

- ▶ **BUSCAR(*clave*):** ¿está *clave* en el árbol?

El TDA Árbol Binario de Búsqueda



Operaciones

- ▶ **BUSCAR(*clave*):** ¿está *clave* en el árbol?
- ▶ **INSERTAR(*clave*):** agregar *clave* manteniendo la propiedad BST.

El TDA Árbol Binario de Búsqueda



Operaciones

- ▶ **BUSCAR(*clave*):** ¿está *clave* en el árbol?
- ▶ **INSERTAR(*clave*):** agregar *clave* manteniendo la propiedad BST.
- ▶ **ELIMINAR(*clave*):** quitar *clave* manteniendo la propiedad BST.

El TDA Árbol Binario de Búsqueda



Operaciones

- ▶ **BUSCAR(*clave*):** ¿está *clave* en el árbol?
- ▶ **INSERTAR(*clave*):** agregar *clave* manteniendo la propiedad BST.
- ▶ **ELIMINAR(*clave*):** quitar *clave* manteniendo la propiedad BST.
- ▶ **MÍNIMO() / MÁXIMO():** el menor/mayor valor almacenado.

El TDA Árbol Binario de Búsqueda



Operaciones

- ▶ **BUSCAR(*clave*):** ¿está *clave* en el árbol?
- ▶ **INSERTAR(*clave*):** agregar *clave* manteniendo la propiedad BST.
- ▶ **ELIMINAR(*clave*):** quitar *clave* manteniendo la propiedad BST.
- ▶ **MÍNIMO() / MÁXIMO():** el menor/mayor valor almacenado.
- ▶ **SUCESOR(*x*) / PREDECESOR(*x*):** el siguiente menor/mayor valor, en orden.

El TDA Árbol Binario de Búsqueda



Operaciones

- ▶ **BUSCAR(*clave*):** ¿está *clave* en el árbol?
- ▶ **INSERTAR(*clave*):** agregar *clave* manteniendo la propiedad BST.
- ▶ **ELIMINAR(*clave*):** quitar *clave* manteniendo la propiedad BST.
- ▶ **MÍNIMO()** / **MÁXIMO():** el menor/mayor valor almacenado.
- ▶ **SUCESOR(*x*)** / **PREDECESOR(*x*):** el siguiente menor/mayor valor, en orden.
- ▶ **RECORRIDO-INORDER():** todos los elementos, en orden creciente.

¡Piénsalo! ¿Cómo se implementarían?



Antes de la próxima clase

¡Piénsalo! ¿Cómo se implementarían?



Antes de la próxima clase

- ▶ **BUSCAR:** en cada nodo, compara con *clave* — ¿izquierda, derecha, o encontrado?

¡Piénsalo! ¿Cómo se implementarían?



Antes de la próxima clase

- ▶ **BUSCAR:** en cada nodo, compara con *clave* — ¿izquierda, derecha, o encontrado?
- ▶ **MÍNIMO:** desde la raíz, siempre ve a la izquierda hasta que no puedas más. ¿Por qué funciona eso?

¡Piénsalo! ¿Cómo se implementarían?



Antes de la próxima clase

- ▶ **BUSCAR:** en cada nodo, compara con *clave* — ¿izquierda, derecha, o encontrado?
- ▶ **MÍNIMO:** desde la raíz, siempre ve a la izquierda hasta que no puedas más. ¿Por qué funciona eso?
- ▶ **INSERTAR:** ¿en qué se parece a **BUSCAR**?

¡Piénsalo! ¿Cómo se implementarían?



Antes de la próxima clase

- ▶ **BUSCAR:** en cada nodo, compara con *clave* — ¿izquierda, derecha, o encontrado?
- ▶ **MÍNIMO:** desde la raíz, siempre ve a la izquierda hasta que no puedas más. ¿Por qué funciona eso?
- ▶ **INSERTAR:** ¿en qué se parece a **BUSCAR**?

La implementación completa, con código real, la construimos en la próxima clase.

Complejidad: todo depende de la altura



El patrón común

BUSCAR, INSERTAR, ELIMINAR, MÍNIMO, MÁXIMO, SUCESOR y PREDECESOR son **todas** $O(h)$, donde h es la **altura** del árbol — cada uno baja, a lo más, un camino de raíz a hoja.

Complejidad: todo depende de la altura



El patrón común

BUSCAR, INSERTAR, ELIMINAR, MÍNIMO, MÁXIMO, SUCESOR y PREDECESOR son **todas** $O(h)$, donde h es la **altura** del árbol — cada uno baja, a lo más, un camino de raíz a hoja.

¡Piénsalo!

¿Cuál es la altura **mínima** posible para n elementos? ¿Y la **máxima**?

Complejidad: todo depende de la altura



El patrón común

BUSCAR, INSERTAR, ELIMINAR, MÍNIMO, MÁXIMO, SUCESOR y PREDECESOR son **todas** $O(h)$, donde h es la **altura** del árbol — cada uno baja, a lo más, un camino de raíz a hoja.

¡Piénsalo!

¿Cuál es la altura **mínima** posible para n elementos? ¿Y la **máxima**?

Respuesta

Mínima: $\Theta(\lg n)$ (árbol lo más “ancho” posible). Máxima: $\Theta(n)$ — exactamente el árbol **degenerado** de hace un rato.

El talón de Aquiles



¿Cuándo se degenera un BST?

Si insertas los elementos ya **ordenados** $(1, 2, 3, \dots, n)$, cada nuevo elemento es siempre mayor que todos los anteriores: se va todo a la derecha, formando una **lista enlazada**, no un árbol.

El talón de Aquiles



¿Cuándo se degenera un BST?

Si insertas los elementos ya **ordenados** $(1, 2, 3, \dots, n)$, cada nuevo elemento es siempre mayor que todos los anteriores: se va todo a la derecha, formando una **lista enlazada**, no un árbol.

En ese caso, $O(h) = O(n)$ — **peor** que un arreglo ordenado. El orden de **inserción** importa muchísimo, y eso es algo que un BST “naive” no controla por sí mismo.

Comparando lo que ya conocemos



Buscar

Insertar

Eliminar

¿Orden?

Comparando lo que ya conocemos



	Buscar	Insertar	Eliminar	¿Orden?
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$	Sí

Comparando lo que ya conocemos



	Buscar	Insertar	Eliminar	¿Orden?
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$	Sí
Lista enlazada	$O(n)$	$O(1)^*$	$O(1)^*$	No

Comparando lo que ya conocemos



	Buscar	Insertar	Eliminar	¿Orden?
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$	Sí
Lista enlazada	$O(n)$	$O(1)^*$	$O(1)^*$	No
Tabla hash	$O(1)$ esp.	$O(1)$ esp.	$O(1)$ esp.	No

Comparando lo que ya conocemos



	Buscar	Insertar	Eliminar	¿Orden?
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$	Sí
Lista enlazada	$O(n)$	$O(1)^*$	$O(1)^*$	No
Tabla hash	$O(1)$ esp.	$O(1)$ esp.	$O(1)$ esp.	No
BST	$O(h)$	$O(h)$	$O(h)$	Sí

Comparando lo que ya conocemos



	Buscar	Insertar	Eliminar	¿Orden?
Arreglo ordenado	$O(\lg n)$	$O(n)$	$O(n)$	Sí
Lista enlazada	$O(n)$	$O(1)^*$	$O(1)^*$	No
Tabla hash	$O(1)$ esp.	$O(1)$ esp.	$O(1)$ esp.	No
BST	$O(h)$	$O(h)$	$O(h)$	Sí

*dado el puntero al nodo. El BST es el **único** que promete orden **y** operaciones sub-lineales — siempre que h se mantenga chico.

Cierre de la sesión



- ▶ Un **árbol** generaliza la lista enlazada permitiendo varios hijos por nodo; un **árbol binario** limita eso a exactamente 2 (izquierdo/derecho).
- ▶ El recorrido **inorder** de cualquier árbol binario visita izquierda, raíz, derecha — y en un BST eso da los elementos en orden creciente, **siempre**.
- ▶ La **propiedad BST**: en cada nodo, izquierda $\leq$ nodo $\leq$ derecha — permite buscar como en búsqueda binaria, pero sobre una estructura enlazada.
- ▶ Todas las operaciones del TDA BST son $O(h)$, con h la altura — $\Theta(\lg n)$ en el mejor caso, $\Theta(n)$ si el árbol se **degenera** (p. ej., insertando datos ya ordenados).
- ▶ El BST es la única estructura vista hasta ahora que combina operaciones sub-lineales **con** orden.

Logros de la sesión



Logros de la sesión



- ▶ Sabemos qué es un árbol, un árbol binario, y sus tres recorridos clásicos.

Logros de la sesión



- ▶ Sabemos qué es un árbol, un árbol binario, y sus tres recorridos clásicos.
- ▶ Sabemos verificar si un árbol cumple la propiedad BST.

Logros de la sesión



- ▶ Sabemos qué es un árbol, un árbol binario, y sus tres recorridos clásicos.
- ▶ Sabemos verificar si un árbol cumple la propiedad BST.
- ▶ Conocemos el TDA BST completo: qué operaciones ofrece, y su complejidad en términos de la altura h .

Logros de la sesión



- ▶ Sabemos qué es un árbol, un árbol binario, y sus tres recorridos clásicos.
- ▶ Sabemos verificar si un árbol cumple la propiedad BST.
- ▶ Conocemos el TDA BST completo: qué operaciones ofrece, y su complejidad en términos de la altura h .
- ▶ Sabemos identificar cuándo un BST se degenera, y por qué eso es un problema.

Gracias